22. Search Engine / Log Search

> **Interviewer:** Design a search engine — think "search my application logs across the whole company," or a product search box. A user types free text like `error database timeout`, and we need to return the most relevant matching documents out of **billions**, fast. Walk me through it.

Candidate: This is a different animal from the usual CRUD systems — the core challenge isn't "store and fetch by key," it's **"find documents by arbitrary words inside them, ranked by relevance, over a dataset too big for one machine."** Let me clarify first.

---

1. Clarifying the requirements

> **Candidate:** 1. Web/product search (write-rarely, read-constantly), or log search (write enormously, read occasionally)? The ratio changes the design. 2. Do we need relevance ranking, or is "any document containing these words" enough? 3. Must results be searchable instantly, or is a short delay acceptable? 4. Filters — e.g. `service = checkout`, `timestamp > yesterday` — alongside free text?
>
> **Interviewer:** Log search — think Elasticsearch/Splunk over app logs. Tens of millions of lines/sec company-wide; engineers search occasionally, especially mid-incident. We need relevance ranking, a couple seconds of indexing delay is fine, and yes — filters by service and time range alongside free text.

Candidate: Requirements, then.

Functional - `POST /_bulk` — ingest documents (log lines) with text + structured fields (service, timestamp, tags). - `GET /search?q=...&filters=...` — free-text query, ranked by relevance, filters, pagination. - Multi-word queries (AND/OR) and phrase queries.

Non-functional - **Write volume is enormous** — tens of millions of lines/sec, dwarfing query volume. - **Query latency stays low** — sub-second, even across billions of documents. - **Dataset too big for one machine** — must shard for storage and query parallelism. - **Near-real-time is fine** — 1-2s indexing delay is acceptable; no synchronous strong write consistency needed. - **Relevance, not just matching** — ranked by a real scoring function.

---

2. Estimation — the number that defines the design

```
10M log lines/sec company-wide, ~300 bytes/line average
  raw ingest ~= 10M * 300B = 3 GB/sec
  per day: 3 GB/sec * 86,400s ~= 260 TB/day  (!)

-> Impossible on one machine's disk, let alone RAM.
-> MUST shard the index across many nodes.
-> MUST tier data: hot window (last 7-30 days) in the fast
   search tier; older data to cold storage, re-indexed on demand.

An inverted index (Section 4) compresses to ~10-30% of raw text
size -> still tens of TB/day. Confirms: many shards, not a bigger box.

Query volume is comparatively tiny: hundreds-to-low-thousands of
queries/sec company-wide, with SPIKES mid-incident (everyone
searching the same error at once).

=> Write path needs a buffered, horizontally-scalable ingest
   pipeline. Read path needs a sharded index with fast fan-out
   (scatter-gather) and hot-query caching.
```

Candidate: Two hard parts: **ingesting a firehose**, and **searching by content instead of by key**. The second — the inverted index — is the conceptual core.

---

3. The naive V1 (and why it dies)

> **Candidate:** Start naive to anchor why it's wrong: dump every log line into a relational table and search with SQL: `SELECT * FROM logs WHERE text LIKE '%timeout%' AND service = 'checkout';`

Why it dies: a leading-wildcard `LIKE` can't use a B-tree index (built for prefix/equality, not "contains anywhere") — the DB falls back to scanning every row. Over billions of rows that's a full table scan per query, getting *slower*, not faster, as data grows. **We need a structure built for "which documents contain this word," not "scan and check."**

---

4. The key idea — the inverted index

> **Candidate:** The trick every real search engine (Lucene/Elasticsearch/Solr) uses: flip "document → words it contains" into "**word → which documents contain it**." That flip is why it's called *inverted*.

```
doc1: "connection timeout on checkout service"
doc2: "database timeout during payment"
doc3: "checkout service returned 500 error"

Inverted index (term -> posting list of doc ids):
  "timeout"  -> [doc1, doc2]
  "checkout" -> [doc1, doc3]
  "database" -> [doc2]
  "service"  -> [doc1, doc3]
  "error"    -> [doc3]
  "payment"  -> [doc2]

Search "timeout" = ONE lookup -> [doc1, doc2]. No scan of 3B docs.
```

Building it (the "analyze" step) — tokenize and normalize raw text into terms:

```
"Connection Timeout on Checkout-Service!"
  -> tokenize:    ["Connection","Timeout","on","Checkout","Service"]
  -> lowercase/strip punctuation, drop stopwords, stem to root:
                  ["connection","timeout","checkout","service"]
These normalized TERMS go into the index, so "TIMEOUTS" still matches.
```

> **Interviewer:** Multi-word queries — how do you combine posting lists, and rank instead of returning an unordered pile?

Candidate: Combine, then **rank**. AND: intersect posting lists (`"database timeout"`); OR: union them. For a phrase like `"checkout service"`, postings must store **term positions** so I can require adjacency, not just co-occurrence.

Rank — BM25 (TF-IDF refined):

```
TF  (term frequency): "timeout" x5 in a doc -> more likely ABOUT timeouts.
IDF (inverse doc freq): a rare term ("checkout") signals more than a
  common one ("service") that's in nearly every line.
score ~= TF(term,doc) x IDF(term,corpus), summed across query terms.
BM25 caps the benefit of very high TF and normalizes for doc length,
so one long doc doesn't win purely by repeating the word.
```

Each shard returns only its **top-K by score**, not the full match set — the key to cheap ranking at scale (Section 6).

---

5. V1 architecture — single-node index

```
   [ Log producers ] --bulk POST--> [ Indexer ] --builds--> [ Inverted Index ]
                                                             (term -> posting list)
                                                                    ^
   [ Engineer's dashboard ] --GET /search?q=...-------------------- |
                                                  returns ranked doc ids + snippets
```

Fine for a demo; falls over at our numbers: 260 TB/day can't live on one disk, and one CPU can't score billions of docs per query in sub-second time. Both **storage** and **query fan-out** must be split.

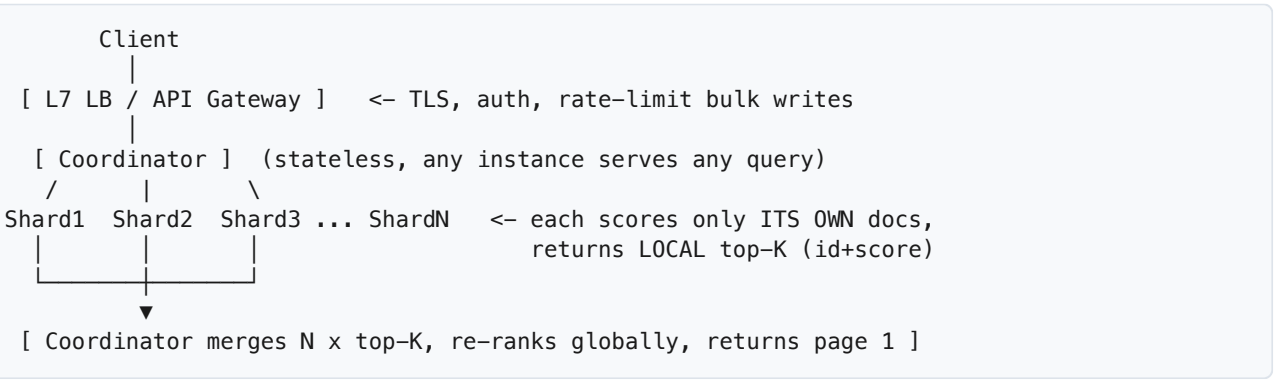
6. Sharding the index + scatter-gather queries

> **Interviewer:** One box can't hold this. How do you shard, and how does a query that might match every shard still come back fast?

Candidate: Two decisions: **split the data**, **fan out the query**.

Sharding: by **document id** (`hash(doc_id) % N`) — each shard holds a complete local inverted index for its own slice. *Not* by term/letter — term frequency is wildly skewed ("error" vs. a rare token), producing lopsided hot shards. Doc-id sharding spreads write and query load evenly.

The catch: a term like `"timeout"` can match docs on every shard, so no single shard has the whole answer. A stateless **coordinator** does **scatter-gather**:



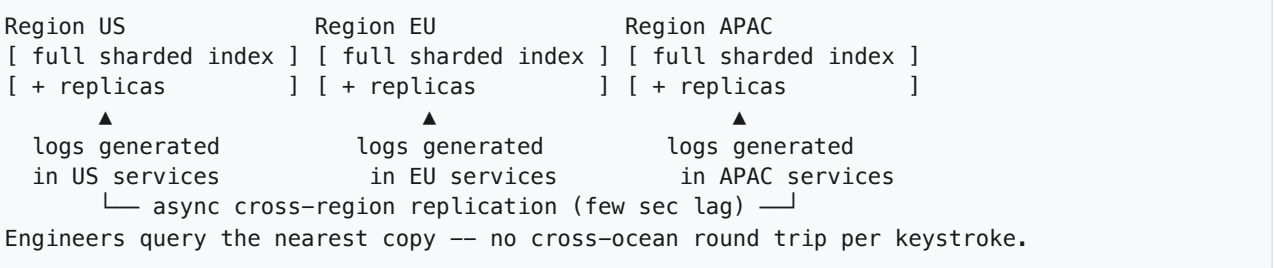
Why local top-K, not everything: if each shard returned every match, the coordinator re-scores billions of rows. Local scoring in parallel across N machines, shipping back only K (~50-100) candidates, makes the merge a cheap `O(N·K log(N·K))` operation.

Why not just an L4/L7 LB: an LB routes one request to one backend; it has no "ask everyone and combine" logic — that fan-out-and-merge is application logic the coordinator provides.

---

7. Multi-region

> **Interviewer:** Engineers in US, EU, APAC all search logs generated in their own regions. What changes?



Logs index **locally first** (lowest latency for the generating region), then replicate asynchronously. A few seconds of cross-region lag is the same near-real-time tradeoff applied globally.

---

8. Latency — where the milliseconds go

> **Interviewer:** Mid-incident, an engineer wants results in well under a second. Where does time go, and how do you cut it?

```
Gateway: TLS + auth                     ~5–10ms
Coordinator: parse query, pick shards   ~1–2ms
Scatter: parallel RPC to N shards       bounded by SLOWEST shard
Each shard: posting lookup + local      ~10–50ms (in-memory postings
  top-K scoring                          for hot/recent data)
Gather: merge N x top-K, re-rank        ~1–5ms  (small lists)
Fetch snippets for top-K only           ~5–10ms (not for every match!)
```

Biggest lever: **fan-out is parallel** — 50 shards costs about as much as 1, bounded by the slowest responder. Second: **keep recent segments in memory** (Section 9) since incident searches mostly hit hot data. Third: never score/fetch full docs for non-candidates — only the merged top-K gets a rendered snippet.

---

9. Consistency — near-real-time indexing

> **Interviewer:** Why can't indexing be instant, and what's happening in that 1-2s window?

```
New log line -> tokenize/normalize -> append to IN-MEMORY segment
  -> periodically (~1s) FLUSH segment to disk as an immutable file
  -> only after flush is the term searchable
  -> small segments periodically MERGED into larger ones in the background
```

- **Buffering before flush** batches many small writes into one efficient disk write.
- **Immutable segments** mean a query never sees a segment mutate mid-read; new data is a new segment, old ones are merged-and-discarded together, never edited in place. This is exactly what lets **indexing and querying run concurrently without locking**.
- **Eventually consistent:** the 1-2s gap between "ingested" and "searchable" — fine for log search; nobody expects to grep a line the instant it's written.

---

10. Async — the ingest pipeline off the write's critical path

> **Interviewer:** 10M lines/sec hitting indexers directly — what if indexing briefly can't keep up?

```
[ App servers/services ] --logs--> [ Kafka: log-lines topic ]
                                     (partitioned, durable, replicated)
                                            |
                                            ▼
                                   [ Indexer consumers ]  (worker pool)
                                            |  tokenize/normalize, write
                                            |  into in-memory segment (Sec.9)
                                            ▼
                                   [ Sharded inverted index ]

                                   ALSO in parallel:
                                   [ Raw log -> object storage / S3 ]
                                   (source of truth; rebuilds a corrupted
                                    segment by re-indexing)
```

- **Kafka absorbs the firehose** — if indexers lag, messages queue rather than get dropped or block the producer.
- **Indexers scale horizontally** behind the buffer, independent of producer burstiness.
- **Raw storage is the real source of truth**; the index is a derived, rebuildable artifact.

---

11. Caching — query cache + the data flow

> **Interviewer:** During an incident, 50 engineers search the same error string within a minute. Do you redo scatter-gather 50 times?

Candidate: No — cache the **final merged result** at the coordinator, keyed by normalized query + filters, short TTL (a few seconds).

```
BUILD PATH: ingest -> Kafka -> tokenize/normalize -> in-memory segment
            -> flush -> disk segment -> (async) merge small into big

QUERY PATH: query -> coordinator
              -> check coordinator result cache (key = query+filters)
                   HIT  -> return cached top-K instantly
                   MISS -> scatter to shards (Sec.6)
                           -> each shard checks ITS OWN cache first
                           -> gather, merge, rank, cache, return
```

Two layers on purpose: **shard-local cache** (hot terms/segments, avoids disk) and **coordinator result cache** (avoids redoing the whole fan-out for a repeated query). A few seconds of staleness is a cheap tradeoff against 50x redundant scatter-gathers at exactly the moment (an incident) load is highest.

---

12. Technology choices — what and why

Concern	Choice	Why this, and why not the alternative
Core index structure	**Inverted index (Lucene-style: Elasticsearch/Solr)**	Precomputes term → posting-list so a query is a lookup, not a scan — the one idea that makes sub-second search over billions of docs possible.

Concern	Choice	Why this, and why not the alternative
Why not SQL `LIKE '%word%'`	rejected	Leading-wildcard `LIKE` can't use a B-tree index → full table scan, gets slower as data grows. Catastrophic at billions of rows.
Sharding strategy	**Hash by document id, N shards, each a full local index**	Spreads write/query load evenly. *Not* sharding by term/letter — term frequency is skewed ("error" vs. a rare token), producing lopsided hot shards.
Query fan-out	**Coordinator doing scatter-gather**	No shard has the whole answer; coordinator asks all in parallel, each returns local top-K, coordinator merges. *Not* an L4/L7 LB alone — it can't fan-out-and-merge.
Ranking	**BM25 (TF-IDF refined)**	Rewards terms frequent-in-doc + rare-overall, length-normalized. *Not* "first N in arbitrary order" — ignores relevance, which requirements need.
Ingest buffering	**Kafka in front of indexers**	Durable, partitioned, absorbs a 10M/s firehose; indexers consume at a sustainable rate. *Not* writing directly into the index — a slow flush would backpressure all the way to producers.
Source of truth for raw docs	**Object storage (S3-style), keyed by id**	Cheap, durable, lets a corrupted/re-tokenized segment be rebuilt by replay. *Not* the index itself — it's a derived, optimized structure, not a durable record.
Segment mutability	**Immutable segments, periodically merged**	Lets indexing and querying run concurrently with zero locking. *Not* in-place updates — that requires locking readers out during writes.
Caching	**Two-tier: shard-local + coordinator result cache, short TTL**	Absorbs repeated-query storms (everyone searching the same error mid-incident). *Not* skipping caching — that's exactly when redundant fan-out would compound load.

Say-it-out-loud justification: "*A relational `LIKE` scan can't use a normal index and gets slower as data grows, so instead I build an inverted index — term to posting list — sharded by document id, queried via a coordinator that scatter-gathers across shards and merges each shard's local top-K by BM25 score; writes flow through Kafka into immutable, periodically-merged segments so indexing and querying never block each other, and a short-TTL cache at both the shard and coordinator level absorbs repeated-query storms during incidents.*"

---

13. Failure modes & graceful degradation

Failure	What happens	Mitigation
Shard down/unreachable during scatter	Coordinator lacks its matches	Return **partial results**, clearly labeled ("18 of 20 shards responded") — never block the whole query
Coordinator dies	In-flight queries fail	Stateless coordinators; LB routes to another instance
Indexer falls behind ingest	Docs searchable later than usual	Kafka buffers the backlog; near-real-time SLA stretches, no data lost

Failure	What happens	Mitigation
Segment file corrupted	That segment unreadable	Rebuild by re-indexing from raw document store (source of truth)
Kafka partition lag spikes	Some shards' new data delayed	Backpressure stays inside the pipeline; producers never blocked
Whole region down	Local searches fail	Reroute to nearest healthy region's full index copy (Section 7)

Design principle: a fast, partial, clearly-labeled answer beats a slow or failed one.

---

14. Follow-up curveballs

> **Interviewer:** How would you support typo tolerance — someone searches "tiemout"?

Candidate: Layer **fuzzy match** on top of the exact-match index: an auxiliary edit-distance or n-gram index over terms finds dictionary terms within 1-2 edits of the query term, then looks those up in the normal posting lists and merges. An extra hop, not a replacement — exact match is cheaper and tried first.

> **Interviewer:** How do you keep ranking fast merging results from hundreds of shards, not three?

Candidate: Same principle, it just scales: each shard **only ever returns local top-K** (say 50), regardless of shard count. The coordinator merges `shards × K` small sorted lists — cheap and bounded — never re-scoring the full corpus. Merge cost grows linearly in `shards × K`, not in total document count.

> **Interviewer:** Leadership wants a live dashboard: "error rate over the last hour" from these logs.

Candidate: That's an **aggregation query**, a different access pattern from search-and- rank. I'd compute it via a background rollup / stream job reading the same Kafka topic, maintaining pre-aggregated counters in a time-series store, and have the dashboard read that — never let a rolling analytics query compete with p99-sensitive interactive searches for the same index resources.

> **Interviewer:** A log line gets ingested twice due to a producer retry — now what?

Candidate: Indexing is an **upsert keyed by document id** — reprocessing the same document overwrites its postings with identical data, no duplicates in any posting list. That's exactly why the bulk ingest API is safe to retry wholesale.

---

15. Deep-dive: "Why not SQL `LIKE` or a database full-text column instead of a dedicated inverted index?"

> **Interviewer:** We already have Postgres/MySQL storing the logs. Why not just add a `tsvector` / `FULLTEXT` column and index that, instead of standing up a whole separate search system? Isn't that one less moving part?

Candidate: Three separate questions get conflated here — "can it find matches at all," "can it rank them," and "can it do either at our scale" — and the three options answer them very differently.

Plain `LIKE '%term%'` is disqualified immediately: a leading wildcard can't use a B-tree (B-trees are built for prefix/equality lookups — `col = 'x'` or `col LIKE 'x%'`), so the planner falls back to a sequential scan of every row, every query, getting slower as the table grows. Section 3 already killed this for V1.

A database's own full-text column (Postgres `tsvector` + GIN, MySQL `FULLTEXT`) is a more serious contender — and worth being precise about *why* it's still not what we'd reach for at this scale, because "it's just `LIKE` again" is the wrong objection:

```
SQL LIKE '%timeout%':
  B-tree index: built for prefix/equality, not "contains anywhere"
    "%timeout%" -> leading wildcard defeats the B-tree -> SEQ SCAN every row
    1B rows -> ~1B comparisons per query, and it only gets worse as data grows

DB full-text (Postgres tsvector + GIN):
  IS an inverted index under the hood -> "timeout" -> posting list, real index lookup
  BUT: lives on ONE Postgres cluster; no built-in scatter-gather across shards
  ts_rank is basically TF-based, no BM25-style length normalization / IDF tuning
  maintaining the GIN index rides the SAME write path as the OLTP rows it indexes

Dedicated inverted index (Lucene/Elasticsearch/Solr):
  Purpose-built: horizontally sharded + replicated, coordinator scatter-gather,
  BM25 ranking, pluggable analyzer pipeline (stemmers, synonyms, fuzzy/edit-distance)
  decoupled entirely from any OLTP write path (Section 10's Kafka buffer)
```

So the honest ranking: `LIKE` can't even find matches efficiently; DB full-text *can* find matches efficiently (it's a real inverted index) but wasn't built to rank well or to scale past one node's disk and CPU; a dedicated search engine is built for exactly the ranking and horizontal-scale problem we estimated in Section 2 (260 TB/day, sub-second queries over billions of docs).

Concern	SQL `LIKE '%term%'`	DB full-text (`tsvector`/GIN, `FULLTEXT`)	Dedicated inverted index (Lucene/ES)
Uses an index at all?	No — leading wildcard defeats the B-tree, full scan	Yes — GIN/FULLTEXT *is* an inverted index	Yes — its entire reason to exist
Relevance ranking	None — no notion of "more relevant"	Basic (`ts_rank`), TF-ish, no length norm	BM25 — TF x IDF, length-normalized, tunable
Tokenization / stemming / stopwords	None — literal substring match only	Limited, built-in dictionaries, hard to extend	Rich analyzer pipeline: stemmers, synonyms, custom tokenizers per field

Concern	SQL `LIKE '%term%'`	DB full-text (`tsvector`/GIN, `FULLTEXT`)	Dedicated inverted index (Lucene/ES)
Fuzzy / typo tolerance	No	Minimal (`pg_trgm` bolt-on)	Native edit-distance / n-gram support
Horizontal scale	Scales with the DB (usually one writer)	Scales with the DB — one node's disk/CPU ceiling	Built to shard + replicate across many nodes
Query fan-out (scatter-gather)	N/A	Not built in — you'd hand-roll it	Native (coordinator + shards, Section 6)
Write-path coupling	Same table, same transactions	Same DB, GIN index maintenance competes with OLTP writes	Decoupled — ingest buffers through Kafka (Section 10)
Right fit for	Simple equality/prefix filters	Bolting light search onto an app that's *mostly* transactional, at modest scale	Search/log workloads that are write-heavy, huge, and need real ranking

> **Rule of thumb:** `LIKE` is disqualified on correctness-of-performance grounds — it simply can't use an index for "contains anywhere." DB full-text is a legitimate choice *if* your data and query volume fit on one node and ranking quality doesn't matter much. The moment you need BM25-quality ranking, tokenization/stemming pipelines, or horizontal scale — reach for a purpose-built inverted-index engine instead of stretching the OLTP database past its design intent.

---

16. Deep-dive: sharding the index by document vs. by term — and the deep-pagination problem

> **Interviewer:** Section 6 sharded by document id and scatter-gathered every query to every shard. Walk me through the alternative — sharding by *term* — and then tell me what breaks when someone asks for page 20 of the results, not page 1.

Candidate: Two different ways to cut up the same inverted index, and they push the cost to completely different places.

Document-based sharding (what we chose) — recap in one line

`hash(doc_id) % N`: every shard holds a *complete* local inverted index for its own slice of documents. A query for `"timeout"` might match docs on *every* shard, so every query fans out to *all* N shards — but each shard's local work (posting lookup, local scoring) is cheap and perfectly parallel.

Term-based sharding — the alternative

Instead, split the *vocabulary*: shard 1 owns terms `a–h`, shard 2 owns `i–p`, shard 3 owns `q–z`, etc. Each shard holds the *complete* posting lists for its terms, across *all* documents.

```
DOCUMENT SHARDING                          TERM SHARDING
query: "database AND timeout"              query: "database AND timeout"

  Coordinator                                Coordinator
   /   |   \                                  /        \
Shard1 Shard2 Shard3  <- ALL shards get     Shard("d") Shard("t")  <- only the
  |     |     |           the query,          holds "database"  holds "timeout"
  |     |     |           each intersects      posting list       posting list
  |     |     |           "database" AND       (maybe millions      (maybe millions
  |     |     |           "timeout" LOCALLY     of doc ids)          of doc ids)
  local  local  local     among ITS OWN docs        \                  /
  top-K  top-K  top-K                                 \                /
    \     |     /                                    SHIP BOTH FULL POSTING
     merge, rank                                     LISTS across the network
     global top-K                                    to intersect them somewhere
                                                      -> huge transfer for common
                                                         terms ("error", "service")
```

Why term sharding is rarely used in practice: a single-term query *is* cheap (one shard answers alone) — but our workload is dominated by multi-term AND/OR queries, and those need posting-list intersection *across* shards, which means shipping potentially enormous posting lists (a common term can appear in tens of millions of docs) over the network just to compute an intersection. Term frequency is also wildly skewed — the shard holding `"error"` or `"timeout"` becomes permanently hot, exactly the lopsided-shard problem Section 6 already rejected term sharding for. Document sharding trades "fan out to everyone" (cheap, parallel) for avoiding "ship giant posting lists between shards" (expensive, serial) — a clearly better trade at our scale.

The problem that survives either way: deep pagination

Even with document sharding, merging is only *exactly* correct for **page 1**. Here's why it breaks past that.

```
3 shards, query "timeout", each returns its LOCAL top-10 by score:

  Shard1 local top-10: [.. scores 99..80 ..]
  Shard2 local top-10: [.. scores 97..75 ..]
  Shard3 local top-10: [.. scores 95..70 ..]

Coordinator merges all 30 candidates, takes GLOBAL top-10 -> CORRECT.
Every doc that could possibly be in the true top-10 was in SOME shard's
top-10, because no shard has a doc scored higher than its own top-10 that
it withheld.

Now ask for PAGE 2 (results #11-20):

  Shard1's 11th-best doc?    We never asked for it -> UNKNOWN, discarded.
  Shard2's 11th-best doc?    Same -> UNKNOWN.
  Shard3's 11th-best doc?    Same -> UNKNOWN.

The true #11-20 globally could easily be Shard2's ranks 2,3,4,5... (Shard2
might just have more good matches than Shard1 or Shard3) — but we only
ever fetched each shard's top-10, so those candidates were never shipped
to the coordinator at all. Page 2 computed from "top-10 per shard" is
WRONG, not just slow.
```

Fix 1 — ask every shard for local top-(offset + page size), every time. To render page 2 (results 11-20) correctly, each shard must return its local top-20, not top-10; page 20 needs local top-200. This is *correct*, but cost grows linearly with depth: at page 1000 every shard computes and ships its local top-

10,000, and the coordinator re-sorts `N * 10,000` candidates for a page nobody actually reads carefully. This is exactly why Elasticsearch caps `from + size` at 10,000 by default — deep offset pagination is fundamentally expensive at this architecture, not a tuning problem.

Fix 2 — cursor-based pagination (`search_after` / keyset pagination). Instead of an offset, the client passes back the **sort key of the last result it saw** (score + doc id as tie-breaker). Each shard seeks directly to "give me the next K docs after this key" — no shard ever recomputes or re-ships everything before the cursor. This turns "page 1000" from "recompute top-10,000" into "resume from a bookmark," and it composes cleanly with immutable segments (Section 9) since the ordering is stable. The tradeoff: you can page forward but not jump arbitrarily to "page 47."

Fix 3 (recommended) — cap default pagination depth, offer a scroll/cursor API for the rare deep case. In practice almost nobody reads past page 3-5 of search results interactively — so serve normal `from` / `size` pagination cheaply with a hard cap (e.g. 10k results, matching Fix 1's honest limit), and for the legitimate deep-scroll use case (an incident responder exporting *every* matching log line) offer a separate scroll/point-in-time cursor endpoint built on Fix 2's keyset approach, which walks the whole index in bounded, resumable chunks instead of pretending it's computing a ranked "page."

> **What to say out loud:** "*I shard by document id, not by term — term sharding looks appealing for single-term queries but falls apart on multi-term AND/OR queries, which need full posting-list intersection shipped across the network, and it re-introduces the hot-shard skew we're trying to avoid. Document sharding's scatter-gather is exactly correct for page 1 because every shard's local top-K is guaranteed to contain anything that could be globally top-K — but that guarantee doesn't extend to page 2 onward, since a shard's rank-11 doc was never fetched. I'd cap default pagination depth (matching Elasticsearch's own `from + size` limit), and for the rare true deep-scroll need — like exporting every log line for an incident — offer a cursor/`search_after`-style API that resumes from a bookmark instead of recomputing an ever-larger top-K on every request.*"

Summary — the mental model

A search engine over billions of documents is **not "look up a row by key" — it's "find documents by arbitrary contained words, ranked by relevance, at a scale no single disk or CPU can hold."** The winning idea is the **inverted index** (term → posting list), turning "scan every document" into "one lookup per query term." Because the index is too big for one node, **shard by document id** and use a **coordinator doing scatter-gather**: fan out to every shard, each scores only its own slice and returns a small local top-K by **BM25**, then merge and re-rank the small combined set. Writes flow through **Kafka** into an indexer that buffers into **immutable, periodically-merged segments** — what lets high-volume indexing and low-latency querying happen concurrently without locking. A short-TTL cache at both the shard and coordinator level absorbs repeated-query storms, and — the guiding principle — a fast, clearly partial answer always beats a slow or failed one.